

COL380 Assignment 2 Report

Abhishek Amrendra Kumar

1 Implementation Details

1.1 Exact KNN (CUDA)

Each thread processes one point and computes distances to all other points. The top $k + 1$ nearest neighbors are maintained using a linear scan. The time complexity is $O(n^2 \cdot k)$, since for each of the n points we compare against all n points and update the top- k list in $O(k)$ time.

CUDA parallelization is used to handle this computational cost efficiently. Without parallelism, the $O(n^2)$ distance computations (on the order of 10^{10} for $n \sim 10^5$) would be prohibitively slow on a CPU.

1.2 Approximate KNN

The approximate KNN algorithm uses grid-based spatial partitioning of the 3D space. Each point is assigned to a grid cell based on its coordinates. For a query point, the algorithm searches within its own cell and expands outward (increasing radius) until at least k neighbors are found.

Key idea: Reduce the search space from all n points to nearby spatial regions, significantly improving runtime.

1.3 OpenMP in Approx KNN

Parallelization is applied across points:

```
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < n; i++) { ... }
```

Dynamic scheduling is used instead of static or guided scheduling because the work per point is uneven due to adaptive radius expansion. Some points find neighbors quickly, while others require a larger search radius. Dynamic scheduling helps balance the workload across threads.

1.4 KMeans Parallelization

Parallelism is used in two stages:

1.4.1 Centroid Update

```
#pragma omp parallel
{
    #pragma omp for nowait
    for (int i = 0; i < n; i++) { ... }
```

```

    #pragma omp critical
    { ... }
}

```

Each thread maintains local buffers to accumulate partial sums, reducing contention. The `nowait` clause removes unnecessary synchronization after the loop, improving performance. A `critical` section is used only during final aggregation to ensure correctness.

Atomic operations are avoided since frequent updates would lead to high contention.

1.4.2 Histogram Computation

```

#pragma omp parallel for
for (int i = 0; i < n; i++) { ... }

```

Each point processes its own cluster independently. Since there are no shared writes, no synchronization is required.

2 Experimental Results

All timings are averaged over multiple runs.

2.1 Scaling with Dataset Size

n	KNN (s)	Approx KNN (s)	KMeans (s)
10	0.011	0.0019	0.0020
100	0.0117	0.0038	0.0028
1000	0.0316	0.0105	0.0124
10000	0.2488	0.0923	0.2754
100000	13.73	0.614	2.337

Table 1: Runtime scaling with dataset size

Observations:

- KNN runtime grows rapidly due to $O(n^2)$ complexity, reaching ~ 13.7 s for $n = 10^5$.
- Approximate KNN scales significantly better and is ~ 20 x faster than exact KNN for large n .
- KMeans shows moderate scaling and remains significantly faster than exact KNN.

2.2 Effect of Iterations in KMeans

Iterations (T)	Time (s)
10	0.703
15	0.905
20	1.089
25	1.300
30	1.487
35	1.745
40	1.908
45	2.119
50	2.337

Table 2: KMeans runtime vs number of iterations ($n = 10^5$)

Observations:

- KMeans runtime increases approximately linearly with the number of iterations.
- Early stopping can significantly reduce runtime without affecting correctness.

2.3 Accuracy

- Exact KNN and KMeans match the reference output exactly ($\text{MAE} = 0$).
- Approximate KNN produces slightly different results but remains within the allowed threshold ($\text{MAE} < 3$).

3 Observations

- KNN is the most accurate but computationally expensive.
- Approximate KNN provides the best speed-accuracy tradeoff.
- KMeans offers moderate performance with stable clustering-based grouping.

4 Parallelization Strategy

Component	Technique
KNN	CUDA
Approx KNN	OpenMP
KMeans	OpenMP

5 Conclusion

CUDA significantly accelerates KNN, but its quadratic complexity remains a bottleneck. Approximate KNN provides the best tradeoff between speed and accuracy, while KMeans offers a balanced alternative.